

TEAM C — Order Service · Matching Engine · Trading UI

Owner: Abidur Rahman Asif (220042152) Your services: **Order Service (8003)** and **Matching Engine (8004)** — the order lifecycle, the saga, and the order book.

Give this whole file to Claude Code in the repo root and say: "Read `TEAM_C_TRADING_CORE.md` and implement my sections completely. `libs/common/` and the `frontend/` shell already exist — read them, import from them, do not rewrite them."

Your only outbound dependency is two HTTP calls from the Order Service to Team B and Team D (`/internal/reservations`, `/internal/share-reservations`). Both contracts are frozen in \$1.9 — build against them and test with the 30-line fake in \$6. You are never blocked.

SECTION 1 — THE SHARED CONTRACT

(Sections 1.1–1.13 are byte-identical in all four team documents. Never change them unilaterally — message the group first.)

1.1 Who builds what (roughly equal load, everyone owns real services)

	Person	Services owned	Also owns	Est. hours
A	Arham Ibrahim Khan (220042160)	Notification (8007), API Gateway (8000)	Docker Compose + all infra + Postgres replication, <code>/notifications</code> UI, docs & diagrams, final merge	~15.5
B	Arham Apon Utsho (220042153)	User (8001), Account (8002)	login/register/wallet/dashboard UI, <code>scripts/smoke_test.py</code> , market-maker bot	~13.5
C	Abidur Rahman Asif (220042152)	Order (8003), Matching Engine (8004)	orders UI + OrderTicket/OrderBook/RecentTrades components	~13.5
D	Mustain Billah Taj (220042166)	Market Data (8005), Portfolio (8006)	market page + candlestick chart + portfolio UI, candle seeder, mock API	~15

1.2 Ownership map — nobody edits anybody else's files

Path	Owner
<code>libs/common/**</code>	pre-built,

Path	Owner
	frozen (A patches it if needed)
frontend/lib/**, frontend/app/layout.tsx, frontend/app/globals.css, frontend/components/{ui,Toast,Protected,Navbar}.tsx, frontend/package.json, configs	pre-built, frozen
requirements-base.txt, .gitattributes, .dockerignore	pre-built
docker-compose.yml, .env, Makefile, infra/postgres/**, docs/**, README.md	A
services/gateway/**, services/notification-service/**	A
frontend/app/notifications/page.tsx, frontend/components/NotificationBell.tsx	A
services/user-service/**, services/account-service/**	B
frontend/app/{login,register,wallet}/page.tsx, frontend/app/page.tsx, frontend/components/{SymbolTable,DepositForm}.tsx	B
scripts/smoke_test.py, infra/seed/market_maker.py	B
services/order-service/**, services/matching-engine/**	C
frontend/app/orders/page.tsx, frontend/components/{OrderTicket,OrderBook,RecentTrades,OrdersTable}.tsx	C
services/market-data-service/**, services/portfolio-service/**	D
frontend/app/market/[symbol]/page.tsx, frontend/app/portfolio/page.tsx, frontend/components/{CandleChart,HoldingsTable}.tsx	D
infra/seed/seed_market_data.py, frontend/mock-api.py	D

1.3 Ports

Component	Container	Port	Host
Frontend	frontend	3000	3000
API Gateway	gateway	8000	8000
User	user-service	8001	8001
Account	account-service	8002	8002
Order	order-service	8003	8003
Matching Engine	matching-engine	8004	8004
Market Data	market-data-service	8005	8005

Component	Container	Port	Host
Portfolio	portfolio-service	8006	8006
Notification	notification-service	8007	8007
Postgres user / account / order	postgres-user / -account / -order	5432	5433 / 5434 / 5435
Postgres market PRIMARY / REPLICA	postgres-market-primary / -replica	5432	5436 / 5437
Postgres portfolio / notification	postgres-portfolio / -notification	5432	5438 / 5439
Redis	redis	6379	6379
RabbitMQ	rabbitmq	5672 / 15672	5672 / 15672

Matching Engine has no database — the book is in memory, by design (proposal §4).

1.4 Money & quantity rules

- Money is `Decimal`, 4 dp, `ROUND_HALF_UP`. **Never float**. Postgres `NUMERIC(18,4)`.
- Money crosses the wire as a **JSON string**: `"195.5000"`. (Exception: `/api/market/candles` returns numbers.)
- Quantity is a positive **int**. Postgres `BIGINT`.
- Limit price must be a positive multiple of `0.01`.
- Use `common.money: to_money, money_str, validate_price, validate_quantity, notional`.
- Symbols: `common.symbols.SYMBOLS`, `SEED_PRICES`, `normalize_symbol`.

1.5 Auth

JWT HS256, secret `JWT_SECRET` identical in every container. Claims: `sub` (user id UUID string), `email`, `username`, `iat`, `exp`, `type:"access"`. Expiry 1440 min. Gateway validates and forwards `Authorization` unchanged; every service validates again via `common.security.get_current_user`. `/internal/*` requires `X-Internal-Key` (Depends(`require_internal_key`)); the gateway refuses to proxy any path containing `/internal/`.

1.6 Gateway routing table

Strip `/api`, keep the rest of the path and the query string.

Method	Public path	Upstream	Auth
POST	<code>/api/auth/register</code> , <code>/api/auth/login</code> , <code>/api/auth/login-json</code>	user-service	public
GET/PATCH	<code>/api/users/me</code>	user-service	JWT

Method	Public path	Upstream	Auth
GET	<code>/api/account/balance,</code> <code>/api/account/transactions</code>	account-service	JWT
POST	<code>/api/account/deposit</code>	account-service	JWT
POST	<code>/api/orders</code>	order-service	JWT, 30/min
GET	<code>/api/orders,</code> <code>/api/orders/{id}</code>	order-service	JWT
DELETE	<code>/api/orders/{id}</code>	order-service	JWT
GET	<code>/api/book/{symbol}</code>	matching-engine	public
GET	<code>/api/market/symbols,</code> <code>/quote/{s},</code> <code>/candles/{s},</code> <code>/trades/{s}</code>	market-data-service	public
GET	<code>/api/portfolio,</code> <code>/api/portfolio/pnl</code>	portfolio-service	JWT
GET/POST	<code>/api/notifications*</code>	notification-service	JWT
WS	<code>/ws/market</code>	market-data-service	public

Errors everywhere: `{"detail": "message"}` — 400 validation, 401 auth, 403 not yours, 404 missing, 409 business conflict, 429 rate limited, 503/504 upstream.

1.7 Event contract — **you publish most of it**

Exchange `exchange.events` (topic, durable). Dead-letter `exchange.events.dead`. Envelope:

```
{ "event_id": "<uuid4>", "event_type": "trade.executed",
  "occurred_at": "2026-08-08T12:00:00.000000Z", "version": 1, "payload": { } }
```

Routing key	Published by	Consumed by
<code>order.accepted</code>	Order (you)	Matching Engine (you)
<code>order.rejected</code>	Order (you)	Account (B), Portfolio (D), Notification (A)
<code>order.cancel_requested</code>	Order (you)	Matching Engine (you)
<code>order.cancelled</code>	Matching Engine (you)	Order (you), Account (B), Portfolio (D), Notification (A)
<code>order.cancel_rejected</code>	Matching Engine (you)	Order (you)
<code>trade.executed</code>	Matching Engine (you)	Order (you), Account (B), Portfolio (D), Market Data (D), Notification (A)

Five other services are coded against these exact field names. A typo here breaks the whole system.

```
// order.accepted
{"order_id","user_id","symbol","side":"BUY|SELL","order_type":"LIMIT|MARKET",
 "price":"195.5000"|null,"quantity":10,"created_at":"...Z"}
// order.rejected
{"order_id","user_id","symbol","reason","rejected_at"}
// order.cancel_requested
{"order_id","user_id","symbol","side","requested_at"}
// order.cancelled
{"order_id","user_id","symbol","side","cancelled_quantity":7,
 "reason":"USER_REQUEST|IOC_REMAINDER|NO_LIQUIDITY","cancelled_at"}
// order.cancel_rejected
{"order_id","user_id","reason":"NOT_IN_BOOK|ALREADY_FILLED","rejected_at"}
// trade.executed
{"trade_id","symbol","price":"195.5000","quantity":5,
 "buy_order_id","sell_order_id","buyer_user_id","seller_user_id",
 "aggressor_side":"BUY|SELL","buy_order_remaining":0,"sell_order_remaining":3,
 "buy_order_limit_price":"196.0000"|null,"executed_at"}
```

Why the Matching Engine — not the Order Service — publishes `order.cancelled` (a deliberate refinement of the proposal; it goes in the README): only the book knows whether the order was still resting when the cancel arrived. If the Order Service released funds optimistically, a fill landing in the same millisecond would settle against a reservation that no longer exists. Two-phase cancel (`cancel_requested` → engine removes → `cancelled`) makes the compensation race-free.

Your queues:

```
q.matching.order_accepted      <- order.accepted
q.matching.cancel_requested    <- order.cancel_requested
q.order.trade_executed         <- trade.executed
q.order.order_cancelled        <- order.cancelled
q.order.cancel_rejected        <- order.cancel_rejected
```

1.8 Redis keys

```
md:last_price:{SYMBOL}      READ ONLY — reference price for MARKET-order fund
reservation
idem:{service}:{event_id}   SET NX EX 86400 — event idempotency
lock:funds / lock:shares    (B and D use these, not you)
```

1.9 Internal service-to-service REST — **the two calls you make**

```
POST http://account-service:8002/internal/reservations
headers X-Internal-Key: <INTERNAL_API_KEY>
```

```

    body {"order_id","user_id","amount":"1020.0000"}
    201 {"reservation_id","order_id","amount","status":"HELD"}
    409 {"detail":"insufficient buying power"}    503 {"detail":"could not
acquire funds lock"}
    Idempotent on order_id.
POST http://account-service:8002/internal/reservations/{order_id}/release -> 200
{"released":"..."}

POST http://portfolio-service:8006/internal/share-reservations
    body {"order_id","user_id","symbol","quantity":10}
    201 {"reservation_id","order_id","quantity","status":"HELD"}
    409 {"detail":"insufficient shares"}
    Idempotent on order_id.
POST http://portfolio-service:8006/internal/share-reservations/{order_id}/release
-> 200 {"released":10}

```

You provide: GET `order-service:8003 /internal/orders/open` (engine book rebuild) and the public GET `matching-engine:8004 /book/{symbol}?depth=10`.

Use `common.http_client.call_service` — it sets `X-Internal-Key`, retries transport errors, raises `ServiceCallError(status_code, detail)`.

1.10 Every service exposes

GET `/health` → `{"status":"ok","service":"<name>"}`, no DB call. GET `/ready` → DB + Redis + broker, 503 if any down. Swagger `/docs, version="1.0.0"`.

1.11 Python service templates (build context is the REPO ROOT)

```

FROM python:3.11-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=/app:/app/libs
COPY requirements-base.txt ./
COPY services/<NAME>/requirements.txt ./service-requirements.txt
RUN pip install --no-cache-dir -r service-requirements.txt
COPY libs /app/libs
COPY services/<NAME>/app /app/app
COPY services/<NAME>/alembic.ini /app/alembic.ini
COPY services/<NAME>/alembic /app/alembic
COPY services/<NAME>/entrypoint.sh /app/entrypoint.sh
RUN chmod +x /app/entrypoint.sh
EXPOSE <PORT>
CMD ["/app/entrypoint.sh"]

```

`requirements.txt` = `-r ../../requirements-base.txt`. `entrypoint.sh` — **LF only**:

```

#!/bin/sh
set -e

```

```
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port "${SERVICE_PORT}"
```

Matching Engine has no DB — skip the alembic COPY lines and the `alembic upgrade head` line. Alembic `env.py` uses `sync_url(settings.DATABASE_URL)` from `common.db`. **Hand-write** `versions/0001_initial.py`.

1.12 Frontend — the shell is already built

Frozen and ready: `lib/api.ts` (typed client incl. `OrderAPI.place/list/get/cancel`, `MarketAPI.book/trades`), `lib/auth.tsx`, `lib/ws.ts`, `lib/format.ts` (`money`, `price`, `qty`, `toneOf`, `clockTime`, `validatePrice`, `validateQuantity`), `components/ui.tsx` (`Card`, `Button` with `variant="buy"|"sell"`, `Input`, `Select`, `Badge` — already styles every order status —, `Table`, `Empty`, `Spinner`, `ErrorBox`), `components/Toast.tsx`, `Protected.tsx`, `Navbar.tsx`, dark theme tokens.

Use these primitives — do not invent new button/card styles.

Route / component	Owner
<code>app/page.tsx</code> , <code>app/login</code> , <code>app/register</code> , <code>app/wallet</code> , <code>components/{SymbolTable,DepositForm}.tsx</code>	B
<code>app/orders/page.tsx</code> , <code>components/{OrderTicket,OrderBook,RecentTrades,OrdersTable}.tsx</code>	C
<code>app/market/[symbol]/page.tsx</code> , <code>app/portfolio/page.tsx</code> , <code>components/{CandleChart,HoldingsTable}.tsx</code>	D
<code>app/notifications/page.tsx</code> , <code>components/NotificationBell.tsx</code>	A

`OrderTicket`, `OrderBook` and `RecentTrades` **already exist as compiling stubs** so Team D's market page builds from hour 0. **Replace the bodies; never change the paths or the props:**

```
<OrderTicket symbol={string} lastPrice={string | null} onPlaced={() => void} />
<OrderBook symbol={string} onPriceClick={(price: string) => void} />
<RecentTrades symbol={string} />
```

1.13 Merge protocol

- Your branch: **feat/team-c-trading**. A works on `main`.
- Nobody edits `libs/common`, the frontend shell, `docker-compose.yml`, `.env`, or another team's folder.
- Push at least every 3 hours even if incomplete.
- Deliver in your folders: `Dockerfile`, `requirements.txt`, `entrypoint.sh`, `alembic/`, `selfcheck.py`, `SERVICE_NOTES.md`.
- Merge windows: **hour 13** and **hour 18**.

SECTION 2 — YOUR SERVICE 1: MATCHING ENGINE (port 8004, no database)

Build this first. It is the highest-risk piece, it needs nothing from anyone, and a pure-function `match()` can be fully tested in one second without Docker.

2.1 Data structures

```
@dataclass
class BookOrder:
    order_id: str; user_id: str; side: str; price: Decimal; remaining: int
    symbol: str; seq: int; order_type: str

class OrderBook:
    symbol: str
    bids: dict[Decimal, deque[BookOrder]]    # price -> FIFO queue (time
priority)
    asks: dict[Decimal, deque[BookOrder]]
    index: dict[str, BookOrder]              # order_id -> order, O(1) cancel
    recent_trades: deque                     # maxlen=200, powers GET
/book/{s}/trades

    books: dict[str, OrderBook]              # one per symbol from
common.symbols.SYMBOLS
    locks: dict[str, asyncio.Lock]          # one per symbol
```

`best_bid = max(bids) / best_ask = min(asks)` — an $O(\text{levels})$ scan. # ponytail: $O(\text{levels})$ best-price scan; swap for a heap only if the book exceeds ~1000 levels. With 8 symbols and a handful of levels this beats a heap and is impossible to get wrong.

2.2 The matching algorithm — write it exactly like this

```
async def handle_order_accepted(env):
    p = env["payload"]
    symbol = normalize_symbol(p["symbol"])
    async with locks[symbol]:                # the whole match is atomic
w.r.t. cancels
        if p["order_id"] in books[symbol].index or p["order_id"] in seen_orders:
            return                          # duplicate delivery
        seen_orders.add(p["order_id"])
        trades, cancel_event = match(books[symbol], BookOrder(...))
        for t in trades:                    # publish AFTER the book is
consistent
            await broker.publish_event(TRADE_EXECUTED, t)
        if cancel_event:
            await broker.publish_event(ORDER_CANCELLED, cancel_event)
```

```

def match(book, incoming) -> tuple[list[dict], dict | None]:
    trades = []
    opposite = book.asks if incoming.side == "BUY" else book.bids
    price_keys = sorted(opposite) if incoming.side == "BUY" else sorted(opposite,
reverse=True)
    for px in price_keys:
        if incoming.remaining == 0:
            break
        if incoming.order_type == "LIMIT":
            if incoming.side == "BUY" and px > incoming.price: break
            if incoming.side == "SELL" and px < incoming.price: break
        level, skipped = opposite[px], deque()
        while level and incoming.remaining > 0:
            resting = level.popleft()
            if resting.user_id == incoming.user_id:          # self-trade prevention:
SKIP
                skipped.append(resting)
                continue
            qty = min(incoming.remaining, resting.remaining)
            incoming.remaining -= qty
            resting.remaining -= qty
            trades.append(make_trade(book.symbol, px, qty, incoming, resting))
            if resting.remaining > 0:
                level.appendleft(resting)                    # KEEPS its time priority
            else:
                book.index.pop(resting.order_id, None)
        while skipped:                                     # restore skipped,
original order, at the front
            level.appendleft(skipped.pop())
            if not level:
                del opposite[px]

    cancel_event = None
    if incoming.remaining > 0:
        if incoming.order_type == "LIMIT":
            side_map = book.bids if incoming.side == "BUY" else book.asks
            side_map.setdefault(incoming.price, deque()).append(incoming)  # rest
in the book
            book.index[incoming.order_id] = incoming
        else:                                              # MARKET remainder is
never rested
            cancel_event = {"order_id": incoming.order_id, "user_id":
incoming.user_id,
                            "symbol": book.symbol, "side": incoming.side,
                            "cancelled_quantity": incoming.remaining,
                            "reason": "IOC_REMAINDER" if trades else
"NO_LIQUIDITY",
                            "cancelled_at": utcnow_iso()}
    return trades, cancel_event

```

The trade price is always the RESTING order's price — the aggressor gets the price improvement. That is what real exchanges do, and it is why `buy_order_limit_price` is in the event: Account needs it to know how much was over-reserved.

```
def make_trade(symbol, px, qty, incoming, resting):
    buy_o, sell_o = (incoming, resting) if incoming.side == "BUY" else (resting, incoming)
    return {"trade_id": str(uuid4()), "symbol": symbol, "price": money_str(px),
            "quantity": qty,
            "buy_order_id": buy_o.order_id, "sell_order_id": sell_o.order_id,
            "buyer_user_id": buy_o.user_id, "seller_user_id": sell_o.user_id,
            "aggressor_side": incoming.side,
            "buy_order_remaining": buy_o.remaining, "sell_order_remaining":
sell_o.remaining,
            "buy_order_limit_price": money_str(buy_o.price) if buy_o.order_type ==
"LIMIT" else None,
            "executed_at": utcnow_iso()}
```

Compute `*_remaining` **after** decrementing both sides for this fill — note the ordering in `match()`.

2.3 Cancel handling

```
async def handle_cancel_requested(env):
    p = env["payload"]; symbol = normalize_symbol(p["symbol"])
    async with locks[symbol]:
        order = books[symbol].index.pop(p["order_id"], None)
        if order is None:
            return await publish(ORDER_CANCEL_REJECTED,
                                {"order_id": p["order_id"], "user_id": p["user_id"],
                                 "reason": "NOT_IN_BOOK", "rejected_at": utcnow_iso()})
        side_map = books[symbol].bids if order.side == "BUY" else
books[symbol].asks
        level = side_map.get(order.price)
        if level:
            try: level.remove(order)
            except ValueError: pass
            if not level: del side_map[order.price]
        remaining = order.remaining
        await publish(ORDER_CANCELLED, {"order_id": order.order_id, "user_id":
order.user_id,
            "symbol": symbol, "side": order.side, "cancelled_quantity": remaining,
            "reason": "USER_REQUEST", "cancelled_at": utcnow_iso()})
```

2.4 REST endpoints

Method	Path	Response
GET	/book/{symbol}? depth=10	{ "symbol", "bids": [{"price", "quantity", "orders"}], "asks": [...], "best_bid", "best_ask", "spread", "ts"} — bids descending, asks ascending, aggregated per price level
GET	/book/{symbol}/trades? limit=50	recent in-memory trades, newest first
GET	/book	all symbols: best bid / ask / spread — powers the dashboard
GET	/internal/stats	{orders_resting, levels, trades_since_start, uptime_s} — a nice figure for the report

Reading the book must take the symbol lock too, or you can serialise a deque mid-mutation.

2.5 Book rebuild after restart (do this around hour 17 — it turns a known weakness into a strength)

```
GET http://order-service:8003/internal/orders/open (X-Internal-Key)
-> insert each order with remaining = quantity - filled_quantity, ordered by
created_at
```

You own both sides, so this costs no coordination. If the call fails, log a warning and start empty — **never fail to boot**.

2.6 Edge cases — Matching Engine

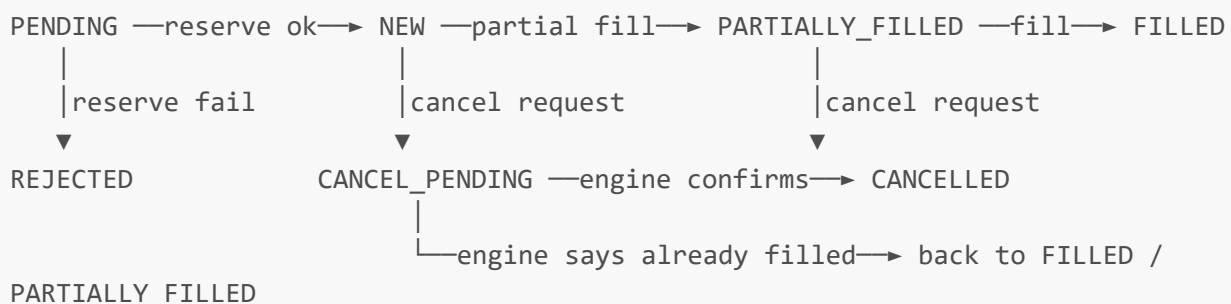
1. **Empty book.** LIMIT rests; MARKET publishes `order.cancelled` with `NO_LIQUIDITY`. Never call `max()` on an empty dict.
2. **Self-trade.** Skipped, not matched. Without this a user trades with themselves, P&L is nonsense, and Account has to lock the same user twice in one event.
3. **A level containing only self-orders.** The skip loop drains the level into `skipped`, restores it, and moves to the next price. **Verify there is no infinite loop — this is the single easiest bug to write here.**
4. **Partially filled resting order** goes back to the **front** (`appendleft`) so it keeps its time priority. `append` silently breaks price–time priority.
5. **Duplicate** `order.accepted` → `order_id` in `index` and a bounded `seen_orders` set, so a redelivered *fully filled* order (no longer in the index) is not matched twice.
6. **Cancel for an order that never existed or already filled** → `cancel_rejected: NOT_IN_BOOK`. Never publish `order.cancelled` for something the book didn't hold, or Account releases funds twice.
7. **Cancel racing a fill.** Both run under the same symbol lock, so one strictly precedes the other. This is exactly why cancel goes through the engine.
8. **await inside the matching loop — don't.** Collect trades into a list, exit the lock, then publish. An `await` mid-match lets the other consumer mutate the book underneath you.
9. **Crossed book invariant.** After every `handle_order_accepted`, assert `best_bid < best_ask` when both exist; log ERROR if violated. Put it behind a `DEBUG_ASSERTS` env flag and leave it on for the demo.

10. **Price as float — never.** Dict keys are `Decimal`; normalise with `to_money` so `100.10` and `100.1` don't split into two levels.
11. **Quantity conservation.** `sum(fill qty) + incoming.remaining == original quantity`, always. Assert it.
12. **MARKET order sweeping several levels** must emit several `trade.executed` events, each with correct running remainders.
13. **Restart loses the book** — documented ceiling + \$2.5 rebuild. # `ponytail: in-memory book by design (proposal §4); rebuilt from Order Service on boot.`
14. **Unknown symbol in an event** → log ERROR, ack, drop. Never create a book for a symbol outside `SYMBOLS`.
15. **Memory** — `recent_trades = deque(maxlen=200)` per symbol, and bound `seen_orders` (e.g. 10 000 entries), or the bot loop grows them forever.

SECTION 3 — YOUR SERVICE 2: ORDER SERVICE (port 8003, `order_db`)

The proposal's saga orchestrator: it owns the order lifecycle and the compensating actions.

3.1 State machine — route **every** status write through one `transition()` helper



Terminal: `FILLED`, `CANCELLED`, `REJECTED`. **A terminal order never changes again.**

3.2 Models

```

class Order(Base):
    __tablename__ = "orders"
    id = UUID pk default uuid4
    user_id = UUID index not null
    client_order_id = String(64) null # client idempotency key
    symbol = String(10) not null index
    side = String(4) not null # BUY | SELL
    order_type = String(8) not null # LIMIT | MARKET
    price = Numeric(18,4) null # null for MARKET
    quantity = BigInteger not null
    filled_quantity = BigInteger not null default 0
  
```

```

    avg_fill_price = Numeric(18,4) not null default 0
    status         = String(20) not null default "PENDING"
    reserved_amount = Numeric(18,4) null          # cash held (BUY) – audit trail
    reject_reason   = String(255) null
    created_at / updated_at
    __table_args__ = (
        UniqueConstraint("user_id", "client_order_id",
name="uq_user_client_order"),
        CheckConstraint("filled_quantity >= 0 AND filled_quantity <= quantity",
name="ck_fill_bounds"),
        CheckConstraint("quantity > 0", name="ck_qty_positive"),
        Index("ix_orders_user_created", "user_id", "created_at"),
    )

class OrderEvent(Base):    # audit trail – screenshot this for the report
    __tablename__ = "order_events"
    id = uuid pk; order_id = UUID index; from_status; to_status; note =
String(255); created_at

class ProcessedEvent(Base):
    event_id = UUID pk; event_type = String(64); handled_at

```

3.3 Endpoints

Method	Path	Notes
POST	/orders	JWT. {symbol, side, order_type, price?, quantity, client_order_id?} → 201
GET	/orders? status=&symbol=&limit=50&offset=0	JWT, own orders only, newest first, limit capped at 200
GET	/orders/{id}	JWT. 404 if it doesn't exist, 403 if it isn't yours
DELETE	/orders/{id}	JWT → 202 {"status": "CANCEL_PENDING"}
GET	/orders/{id}/events	JWT, the audit trail
GET	/internal/orders/open	internal key — every NEW/PARTIALLY_FILLED order, for the engine's book rebuild

3.4 The place-order saga — exactly this order of operations

1. VALIDATE (400 on any failure, nothing persisted)
 - symbol → normalize_symbol (unknown → 400)
 - side → BUY | SELL
 - order_type → LIMIT | MARKET
 - quantity → validate_quantity (positive int, <= 1_000_000)
 - LIMIT → price required, validate_price (>0, multiple of 0.01)
 - MARKET → price MUST be absent/null (400 if supplied)

2. CLIENT IDEMPOTENCY

```
if client_order_id given and (user_id, client_order_id) already exists:
    return that order with 200 – never a second order
```

3. PERSIST as PENDING and COMMIT (we need the order_id before reserving)

4. RESERVE

```
BUY LIMIT : amount = notional(price, quantity)
BUY MARKET: ref = redis md:last_price:{SYM} or SEED_PRICES[SYM]
              if neither -> reject NO_MARKET_PRICE
              amount = to_money(ref * quantity * Decimal("1.05")) # 5%
```

slippage buffer

```
-> POST account-service /internal/reservations
```

```
SELL (both): -> POST portfolio-service /internal/share-reservations
{quantity}
```

```
409 -> status REJECTED, reject_reason = detail, publish order.rejected,
      return 409 to the client with the same detail
```

```
503/timeout -> status REJECTED, reason SERVICE_UNAVAILABLE,
              best-effort compensating release (idempotent, safe even if nothing
was held),
              publish order.rejected, return 503
```

5. transition PENDING -> NEW, store reserved_amount, COMMIT

6. PUBLISH order.accepted

```
on failure: transition NEW -> REJECTED (reason BROKER_UNAVAILABLE),
COMPENSATE by releasing BOTH funds and shares (both idempotent no-ops when
absent),
```

```
return 503. ** This is the saga's compensating action – log it loudly, it is
a graded concept. **
```

7. return 201

3.5 Cancel flow (two-phase — the book is the authority)

```
DELETE /orders/{id}
  not found -> 404
  order.user_id != caller -> 403 (check ownership BEFORE
status)
  status in (FILLED, CANCELLED, REJECTED) -> 409 "order is already <status>"
  status == CANCEL_PENDING -> 409 "cancellation already in
progress"
  transition -> CANCEL_PENDING, publish order.cancel_requested, return 202

consume order.cancelled -> CANCEL_PENDING / NEW / PARTIALLY_FILLED ->
CANCELLED
consume order.cancel_rejected -> revert CANCEL_PENDING -> FILLED if filled ==
```

quantity

else PARTIALLY_FILLED if filled > 0 else NEW

3.6 Fill handling (`trade.executed`)

```
if already_processed(redis, "order", event_id): return
for (order_id, remaining) in [(buy_order_id, buy_order_remaining), (sell_order_id,
sell_order_remaining)]:
    order = SELECT ... FOR UPDATE                # two fills can arrive
back to back
    if order is None: log WARNING and continue    # not ours – never crash
    new_filled = order.filled_quantity + quantity
    if new_filled > order.quantity: log ERROR; new_filled = order.quantity    #
never break the CHECK
    order.avg_fill_price = to_money(
        (order.avg_fill_price * order.filled_quantity + to_money(price) *
quantity) / new_filled)
    order.filled_quantity = new_filled
    transition -> FILLED if new_filled == order.quantity else PARTIALLY_FILLED
insert ProcessedEvent(event_id)
```

Wrap every handler so that on exception it calls `clear_processed(redis, "order", event_id)` before re-raising.

3.7 Edge cases — Order Service

1. **Double-submit from a double-clicked button** → `client_order_id` unique per user returns the same order. Without it the user pays twice.
2. **Reserve succeeded but the process died before publishing** → the order sits in `NEW` with funds held forever. **Startup reconciliation:** on boot, any `PENDING` older than 60 s → `REJECTED` + release; any `NEW` the engine may never have seen → republish `order.accepted` (the engine dedupes by `order_id`).
3. **Account returns 409 after a timeout where the first call actually succeeded** → the endpoint is idempotent on `order_id`, so no double hold. Still call `release` on the failure path; it is a safe no-op.
4. **MARKET order with no reference price** → reject clearly. Never reserve 0.
5. **MARKET order where the 5 % buffer is not enough** → the engine fills what it can, the remainder is cancelled (`IOC_REMAINDER`) and the excess hold released. Document that MARKET is effectively Immediate-Or-Cancel.
6. **price supplied on a MARKET order** → 400. Don't silently ignore it.
7. **Sub-tick price 100.005, "1e5", "NaN", "Infinity", negative, 0** → 400.
8. **Quantity 0, -5, 10.5, "10", true** → 400. `bool` is an `int` subclass in Python — `validate_quantity` already rejects it; don't hand-roll the check.
9. **Unknown or lowercase symbol** → normalize, then 400 if unknown.
10. **Cancelling someone else's order** → 403, no state change.
11. **Cancelling an order that filled 1 ms ago** → `cancel_rejected: ALREADY_FILLED`, order returns to `FILLED`. The client briefly sees `CANCEL_PENDING` — that is correct, not a bug.
12. **Double cancel** → 409.

13. **Fills arriving out of order** (partial after final) → guard `filled_quantity <= quantity`, never leave a terminal state.
14. **Duplicate `trade.executed`** → Redis idem + `ProcessedEvent` PK.
15. **A trade naming an order we don't have** → WARN and ack. Never nack-loop.
16. **avg_fill_price divide-by-zero** when `new_filled == 0` — impossible, guard anyway.
17. **Listing orders** — paginate, cap `limit` at 200, index (`user_id`, `created_at` DESC).
18. **Two fills for the same order concurrently** → one consumer per queue with `prefetch=1`, plus `SELECT FOR UPDATE`. Both.
19. **Broker down at startup** → `Broker.connect` retries 60×2 s; `/ready` returns 503 until connected. Don't serve `POST /orders` and reject everything.
20. **Time-in-force** — all LIMIT orders are Good-Till-Cancelled, MARKET is IOC. State it in `SERVICE_NOTES.md`.

SECTION 4 — YOUR FRONTEND WORK

4.1 `components/OrderTicket.tsx` (stub exists — replace the body, keep path and props)

BUY/SELL toggle (`Button variant="buy" | "sell"`), LIMIT/MARKET toggle, quantity, price (hidden for MARKET, prefilled from `lastPrice` and from `OrderBook`'s `onPriceClick`), estimated cost `price × qty` shown live, available cash from `AccountAPI.balance()` next to it, and for SELL the available share count from `PortfolioAPI.get()` ("10 of 15 — 5 reserved by open orders"). Validate with `validatePrice` / `validateQuantity` from `lib/format` **before** submitting. Generate `client_order_id` once per ticket with `crypto.randomUUID()`. Disable the button while in flight. On success: toast, call `onPlaced()`. On error: show `ApiError.detail` verbatim in an `<ErrorBox>` (409 "insufficient buying power" must be readable).

4.2 `components/OrderBook.tsx`

Poll `MarketAPI.book(symbol, 10)` every 2 s (the book has no WS feed — that is fine and documented). Asks descending on top, bids descending below, spread in the middle, depth bars sized by cumulative quantity, click a row → `onPriceClick(price)`. Handle the empty book with `<Empty>`.

4.3 `components/RecentTrades.tsx`

`MarketAPI.trades(symbol, 30)` on mount, newest first; price coloured by `aggressor_side`; time via `clockTime`. Refresh every 3 s (or prepend from the live tick stream if you have time).

4.4 `app/orders/page.tsx` + `components/OrdersTable.tsx`

`<Protected>`. Two sections: **Open** (`NEW`, `PARTIALLY_FILLED`, `CANCEL_PENDING`) with a Cancel button, and **History** (`FILLED`, `CANCELLED`, `REJECTED`). Columns: time, symbol, side badge, type, price, filled/quantity, avg fill price, status `<Badge>`, action. Poll every 3 s **while any order is non-terminal**, then stop. On cancel → optimistic `CANCEL_PENDING`, toast, refetch. **Render `CANCEL_PENDING` as "Cancelling...", not "Cancelled"** — it can still come back `FILLED`. (`Badge` already does this.) Show `reject_reason` in a tooltip on rejected rows.

SECTION 5 — `main.py` SKELETON

```

setup_logging(settings.SERVICE_NAME, settings.LOG_LEVEL)
engine = make_engine(settings.DATABASE_URL)          # matching-engine: skip entirely
Session = make_sessionmaker(engine)

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = make_redis(settings.REDIS_URL)
    app.state.broker = Broker(settings.RABBITMQ_URL, settings.SERVICE_NAME)
    await app.state.broker.connect()
    await app.state.broker.consume(Q_ORDER_TRADE_EXECUTED, [TRADE_EXECUTED],
    handle_trade_executed)
    await app.state.broker.consume(Q_ORDER_CANCELLED, [ORDER_CANCELLED],
    handle_order_cancelled)
    await app.state.broker.consume(Q_ORDER_CANCEL_REJECTED,
    [ORDER_CANCEL_REJECTED], handle_cancel_rejected)
    await startup_reconciliation()                    # Order Service only, §3.7.2
    yield
    await app.state.broker.close(); await app.state.redis.aclose()

app = FastAPI(title="Order Service", version="1.0.0", lifespan=lifespan)

```

Handlers open their **own** session from `Session()` — never a request-scoped one.

SECTION 6 — LOCAL DEV & TESTING ALONE

```

# services/order-service/docker-compose.dev.yml
services:
  postgres: { image: postgres:16-alpine, environment: {POSTGRES_USER: mse,
  POSTGRES_PASSWORD: mse_pw, POSTGRES_DB: order_db}, ports: ["5435:5432"]}
  redis:    { image: redis:7-alpine, ports: ["6379:6379"]}
  rabbitmq: { image: rabbitmq:3.13-management-alpine, ports:
  ["5672:5672", "15672:15672"]}

```

PowerShell: `$env:PYTHONPATH=".;../..libs"; uvicorn app.main:app --reload --port 8003`

Fake Team B and Team D — `services/order-service/scripts/fake_reservations.py`:

```

from fastapi import FastAPI, HTTPException
app = FastAPI()
HELD, SHARES = {}, {}
@app.post("/internal/reservations", status_code=201)
async def reserve(body: dict):
    if body["order_id"] in HELD: return HELD[body["order_id"]]

```

```

    if float(body["amount"]) > 100000: raise HTTPException(409, "insufficient
    buying power")
    HELD[body["order_id"]] = {"reservation_id": body["order_id"], **body,
    "status": "HELD"}
    return HELD[body["order_id"]]
@app.post("/internal/reservations/{oid}/release")
async def release(oid: str): HELD.pop(oid, None); return {"released": "0.0000"}
@app.post("/internal/share-reservations", status_code=201)
async def sreserve(body: dict):
    if body["quantity"] > 1000: raise HTTPException(409, "insufficient shares")
    SHARES[body["order_id"]] = body
    return {"reservation_id": body["order_id"], **body, "status": "HELD"}
@app.post("/internal/share-reservations/{oid}/release")
async def srelease(oid: str): SHARES.pop(oid, None); return {"released": 0}

```

Run it on 8002 and 8006 and point `ACCOUNT_SERVICE_URL / PORTFOLIO_SERVICE_URL` at it.

`services/matching-engine/selfcheck.py` — pure asserts against `match()`, no broker, no network, runs in a second. **The single most valuable test in the project:**

```

empty book + LIMIT BUY 10@100          -> 0 trades, 1 bid level
then LIMIT SELL 10@100                 -> 1 trade @100, book empty
partial: BUY 10@100 then SELL 4@100    -> 1 trade qty 4, bid remaining 6,
buy_order_remaining 6
price improvement: SELL 10@100 resting, BUY 10@105 -> trade @100 (NOT 105)
time priority: BUY A 5@100, BUY B 5@100, SELL 5@100 -> A fills, B untouched
price priority: BUY 5@101 and BUY 5@100, MARKET SELL 5 -> the 101 bid fills first
self-trade: same user both sides       -> 0 trades, both rest
self-only level: user1 bid 5@100, user1 MARKET SELL 5 -> 0 trades, NO_LIQUIDITY,
no infinite loop
market sweep: asks 5@100, 5@101, 5@102 ; BUY MARKET 12 -> 3 trades (5,5,2),
remaining 0
market no liquidity: empty book, BUY MARKET 5 -> NO_LIQUIDITY, cancelled_quantity
5
cancel resting 10@100                  -> cancelled_quantity 10, index empty
cancel unknown                         -> cancel_rejected NOT_IN_BOOK
INVARIANT every case: sum(trade qty) + remaining == original quantity
INVARIANT every case: best_bid < best_ask whenever both exist

```

SECTION 7 — HOUR-BY-HOUR PLAN (24 h)

Hours	Work
0–1	Pull repo, read <code>libs/common</code> , start dev containers.
1–6	Matching Engine first. Pure <code>match()</code> + <code>selfcheck.py</code> green before you touch RabbitMQ.
6–8	Wire the engine to the broker, REST book endpoints, Dockerfile. Push <code>feat/team-c-trading</code> .

Hours	Work
8–14	Order Service: models, validation, the saga with compensation, two-phase cancel, fill handling, all endpoints. Test against the fake in §6.
14–15	Startup reconciliation + <code>/internal/orders/open</code> + Dockerfile. Push.
15–18	Frontend: <code>OrderTicket</code> , <code>OrderBook</code> , <code>RecentTrades</code> , <code>/orders</code> page. Build against Team D's <code>frontend/mock-api.py</code> if the backend isn't merged. Push — Team A's merge window.
18–21	On call for integration: order → fill → statuses correct end to end. Engine book-rebuild-on-boot if the clock allows.
21–23	<code>SERVICE_NOTES.md</code> ×2. Screenshots: the order book UI, a multi-level market sweep in the logs, the RabbitMQ queue graph, the <code>order_events</code> audit trail.
23–24	Freeze.

SECTION 8 — DEFINITION OF DONE

- ☐ `services/matching-engine/selfcheck.py` passes **every** case in §6, including both invariants.
- ☐ Both services build from the repo root; `/health`, `/ready`, `/docs` green.
- ☐ Placing an order with no funds → 409 `insufficient buying power`, order persisted `REJECTED`, funds released.
- ☐ Two opposing orders from two users fully match; both reach `FILLED`.
- ☐ Cancelling a resting order → `CANCELLED`, with the right `cancelled_quantity` on the event.
- ☐ Cancelling an already-filled order → `cancel_rejected`; the order stays `FILLED`.
- ☐ Every event payload matches §1.7 **field for field**.
- ☐ Replaying any event twice changes nothing.
- ☐ Order ticket, order book, recent trades and the orders page all work against the real gateway.
- ☐ `SERVICE_NOTES.md` for both services, including the stated time-in-force semantics.